

**LAPORAN PRATIUM STRUKTUR DATA**

**IMPLEMENTASI DOUBLE LINKED LIST DALAM JAVA**

DI SUSUN OLEH :

ABDUL KARIM ALGAZALI

NIM 2511532029

DOSEN PENGAMPU : Dr.WAHYUDI, S.T, M.T

ASISTEN LABORATORIUM : Zahran Azaria Anvaya



**DEPARTEMEN INFORMATIKA**

**FAKULTAS TEKNOLOGI INFORMASI**

**UNIVERSITAS ANDALAS**

**PADANG**

**2026**

## KATA PENGANTAR

Puji syukur kehadiran Tuhan Yang Maha Esa, karena atas rahmat dan izin-Nya laporan praktikum “Implementasi Double Linked List dalam Java” ini dapat diselesaikan dengan baik. Saya ucapkan terima kasih sebesar-besarnya kepada Dr. Wahyudi, S.T, M.T selaku dosen pengampu mata kuliah Struktur Data, serta kepada asisten laboratorium yang telah membimbing pelaksanaan praktikum ini.

Laporan ini disusun sebagai bentuk dokumentasi kegiatan praktikum yang bertujuan untuk memahami konsep struktur data Double Linked List (DLL), yang merupakan pengembangan dari Single Linked List dengan penambahan referensi ke node sebelumnya. DLL memungkinkan navigasi dua arah sehingga lebih fleksibel dalam berbagai operasi. Praktikum ini mencakup implementasi kelas node, operasi penyisipan (depan, belakang, posisi tertentu), operasi penghapusan (depan, belakang, posisi tertentu), penelusuran maju dan mundur, serta penerapannya dalam studi kasus playlist musik yang memanfaatkan keunggulan traversal dua arah.

Saya menyadari bahwa penulisan laporan praktikum ini masih jauh dari kata sempurna. Namun, dengan segala kemampuan dan pengetahuan yang dimiliki, saya telah berupaya agar laporan ini dapat selesai dengan baik. Oleh karenanya, saya dengan rendah hati menerima saran, masukan, dan kritikan guna penyempurnaan laporan ini.

Padang, Mei 2026

Penulis

## DAFTAR ISI

|                                                |           |
|------------------------------------------------|-----------|
| <b>KATA PENGANTAR.....</b>                     | <b>i</b>  |
| <b>DAFTAR ISI.....</b>                         | <b>ii</b> |
| <b>BAB I PENDAHULUAN.....</b>                  | <b>3</b>  |
| 1.1 Latar Belakang .....                       | 3         |
| 1.2 Tujuan .....                               | 3         |
| 1.3 Manfaat .....                              | 4         |
| <b>BAB II PEMBAHASAN.....</b>                  | <b>5</b>  |
| 2.1 Konsep Double Linked List dalam Java ..... | 5         |
| 2.2 Langkah pengerjaan .....                   | 6         |
| 2.3 Latihan .....                              | 11        |
| <b>BAB III KESIMPULAN.....</b>                 | <b>13</b> |
| 3.1 Kesimpulan .....                           | 13        |
| 3.2 Saran .....                                | 13        |
| <b>DAFTAR PUSTAKA.....</b>                     | <b>14</b> |

# **BAB I**

## **PENDAHULUAN**

### **1.1 Latar Belakang**

Struktur data Single Linked List (SSL) yang telah dipelajari sebelumnya memiliki kelebihan dalam alokasi memori dinamis dan kemudahan penyisipan/penghapusan di awal list. Namun SSL memiliki kelemahan utama, yaitu hanya memiliki referensi satu arah (ke node berikutnya), sehingga tidak dapat dilakukan navigasi mundur secara efisien. Operasi penghapusan di akhir list masih memerlukan traversal  $O(n)$  kecuali jika disimpan referensi tail (seperti yang dioptimalkan pada praktikum sebelumnya). Untuk mengatasi keterbatasan arah navigasi tersebut, dikembangkan Double Linked List (DLL).

Double Linked List adalah struktur data linier yang setiap nodenya memiliki dua referensi, yaitu next (menunjuk ke node berikutnya) dan prev (menunjuk ke node sebelumnya). Dengan adanya dua pointer ini, DLL memungkinkan traversal dua arah, sehingga operasi seperti penghapusan node terakhir, penelusuran mundur, atau penyisipan sebelum node tertentu dapat dilakukan dengan lebih mudah dan efisien. DLL banyak digunakan dalam aplikasi yang memerlukan navigasi fleksibel, seperti playlist musik (next/previous), undo-redo pada editor, dan browser history.

Pada praktikum Pekan 6 ini, mahasiswa diminta untuk mengimplementasikan kelas node DLL, operasi penyisipan dan penghapusan di berbagai posisi, penelusuran maju-mundur, serta menerapkannya dalam studi kasus sistem playlist musik. Praktikum ini menekankan pada pemahaman pengelolaan dua pointer (next dan prev) serta penanganan kasus-kasus khusus untuk menjaga konsistensi list.

## 1.2 Tujuan

1. Memahami konsep Double Linked List dan perbedaannya dengan Single Linked List.
2. Mampu mengimplementasikan kelas node DLL dengan dua referensi (next dan prev).
3. Mampu mengimplementasikan operasi penyisipan di depan, di belakang, dan di posisi tertentu.
4. Mampu mengimplementasikan operasi penghapusan di depan, di belakang, dan di posisi tertentu.
5. Mampu melakukan penelusuran maju (forward) dan mundur (backward) pada DLL.
6. Menerapkan DLL untuk menyelesaikan studi kasus playlist musik yang membutuhkan navigasi dua arah.

## 1.3 Manfaat

1. Mahasiswa memahami kelebihan DLL dalam traversal dua arah dan efisiensi operasi di kedua ujung list.
2. Mahasiswa terampil mengelola referensi next dan prev untuk menjaga integritas struktur data.
3. Mahasiswa mampu mengidentifikasi dan menangani kondisi batas seperti list kosong, node tunggal, dan operasi di ujung list untuk mencegah NullPointerException.
4. Mahasiswa memperoleh pengalaman membangun aplikasi simulasi nyata yang memanfaatkan struktur DLL sebagai backend penyimpanan data.

## BAB II

### PEMBAHASAN

#### 2.1 Konsep Single Linked List dalam Java

Double Linked List (DLL) adalah struktur data linier yang tersusun dari node-node yang saling terhubung dua arah. Setiap node memiliki tiga komponen:

- Data: Nilai atau objek yang disimpan.
- Next: Referensi (pointer) ke node berikutnya dalam list. Node terakhir memiliki next = null.
- Prev: Referensi (pointer) ke node sebelumnya dalam list. Node pertama memiliki prev = null.

Perbedaan mendasar dengan Single Linked List (SSL) adalah keberadaan pointer prev yang memungkinkan:

- Traversal mundur dari tail ke head.
- Penghapusan node tanpa perlu menyimpan pointer prev secara eksplisit (karena node yang akan dihapus sudah memiliki akses ke node sebelumnya).
- Penyisipan node di akhir tanpa perlu traversal jika disimpan pointer tail.

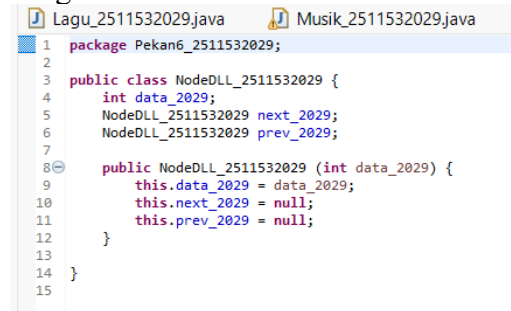
Operasi fundamental pada DLL meliputi:

- Insert at Front: Menyisipkan node baru di awal list. `new_node.next = head`, `head.prev = new_node`, lalu head diperbarui ke `new_node`.
- Insert at End: Menyisipkan node baru di akhir list. Dilakukan traversal hingga node terakhir, lalu `curr.next = new_node` dan `new_node.prev = curr`.
- Insert at Position: Menyisipkan node pada indeks tertentu; perlu traversal ke node sebelum posisi target, lalu menyesuaikan kedua pointer (next dan prev) dari node baru, node sebelumnya, dan node setelahnya.
- Delete Head: Menghapus node pertama. Head dipindahkan ke `head.next`, lalu jika head baru tidak null, `head.prev = null`.
- Delete Last: Menghapus node terakhir. Perlu traversal untuk menemukan node kedua-terakhir, lalu `second_last.next = null`.
- Delete at Position: Menghapus node pada posisi tertentu dengan menyesuaikan pointer prev dan next dari node sekitar.
- Search: Traversal maju (atau mundur) dengan membandingkan data setiap node hingga ditemukan.
- Forward Traversal: Mengunjungi setiap node dari head hingga null.
- Backward Traversal: Mengunjungi setiap node dari tail hingga null.

## 2.2 Langkah pengerjaan

### 1. NodeDLL\_2511532029

Program:



```
1 package Pekan6_2511532029;
2
3 public class NodeDLL_2511532029 {
4     int data_2029;
5     NodeDLL_2511532029 next_2029;
6     NodeDLL_2511532029 prev_2029;
7
8     public NodeDLL_2511532029 (int data_2029) {
9         this.data_2029 = data_2029;
10        this.next_2029 = null;
11        this.prev_2029 = null;
12    }
13 }
14
15 }
```

Gambar 2.1

Penjelasan program:

Program ini mendefinisikan kelas node yang menjadi komponen dasar Double Linked List. Atributnya adalah data\_2029 bertipe integer, next\_2029 bertipe referensi ke NodeDLL\_2511532029 yang menunjuk ke node berikutnya, dan prev\_2029 yang menunjuk ke node sebelumnya. Konstruktor menginisialisasi data dan mengeset kedua pointer ke null. Keberadaan pointer prev inilah yang membedakan DLL dengan SSL yang hanya memiliki pointer next.

## 2. InsertDLL\_2511532029 Program:

```
1 package Pk2511532029;
2
3 public class InsertDLL_2511532029 {
4
5     static NodeDLL_2511532029 insertBegin(NodeDLL_2511532029 head_2029, int data_2029) {
6         NodeDLL_2511532029 new_node_2029 = new NodeDLL_2511532029(data_2029);
7         new_node_2029.next_2029 = head_2029;
8         if (head_2029 != null) {
9             head_2029.prev_2029 = new_node_2029;
10        }
11        return new_node_2029;
12    }
13
14    public static NodeDLL_2511532029 insertEnd(NodeDLL_2511532029 head_2029, int newdata_2029) {
15        NodeDLL_2511532029 newnode_2029 = new NodeDLL_2511532029(newdata_2029);
16        if (head_2029 == null) {
17            return newnode_2029;
18        } else {
19            NodeDLL_2511532029 curr_2029 = head_2029;
20            while (curr_2029.next_2029 != null) {
21                curr_2029 = curr_2029.next_2029;
22            }
23            curr_2029.next_2029 = newnode_2029;
24            newnode_2029.prev_2029 = curr_2029;
25        }
26        return head_2029;
27    }
28
29    public static NodeDLL_2511532029 insertAtPosition(NodeDLL_2511532029 head_2029, int pos_2029, int newdata_2029) {
30        // Case 1: Insert at the beginning (Position 1)
31        if (pos_2029 <= 1) {
32            return insertBegin(head_2029, newdata_2029);
33        }
34
35        NodeDLL_2511532029 new_node_2029 = new NodeDLL_2511532029(newdata_2029);
36        NodeDLL_2511532029 curr_2029 = head_2029;
37
38        // Traverse to the node before the insertion point
39        for (int i = 2; i < pos_2029 + 1; i++) {
40            if (curr_2029 == null) {
41                return head_2029;
42            }
43            curr_2029 = curr_2029.next_2029;
44        }
45
46        // Case 2: Position is out of bounds
47        if (curr_2029 == null) {
48            System.out.println("Posisi tidak ada");
49            return head_2029;
50        }
51
52        // Case 3: Insert in middle or at the very end
53        new_node_2029.next_2029 = curr_2029.next_2029;
54        new_node_2029.prev_2029 = curr_2029;
55
56        if (curr_2029.next_2029 != null) {
57            curr_2029.next_2029.prev_2029 = new_node_2029;
58        }
59        curr_2029.next_2029 = new_node_2029;
60        return head_2029;
61    }
62
63    public static void PrintList_2511532029(NodeDLL_2511532029 head_2029) {
64        NodeDLL_2511532029 curr_2029 = head_2029;
65        while (curr_2029 != null) {
66            System.out.print(curr_2029.data_2029 + (curr_2029.next_2029 != null ? " <-> " : ""));
67            curr_2029 = curr_2029.next_2029;
68        }
69        System.out.println();
70    }
71
72    public static void main(String[] args) {
73        // Manual initialization
74        NodeDLL_2511532029 head_2029 = new NodeDLL_2511532029(2);
75        head_2029.next_2029 = new NodeDLL_2511532029(3);
76        head_2029.prev_2029 = head_2029;
77        head_2029.next_2029.next_2029 = new NodeDLL_2511532029(5);
78        head_2029.next_2029.next_2029.prev_2029 = head_2029;
79
80        System.out.print("List awal : ");
81        PrintList_2511532029(head_2029);
82
83        head_2029 = insertBegin(head_2029, 1);
84        System.out.print("Simbol 1 ditambahkan di awal : ");
85        PrintList_2511532029(head_2029);
86
87        System.out.print("Simbol 6 ditambahkan diakhir : ");
88        head_2029 = insertEnd(head_2029, 6);
89        PrintList_2511532029(head_2029);
90
91        System.out.print("Tambah node 4 di posisi 4 : ");
92        head_2029 = insertAtPosition(head_2029, 4, 4);
93        PrintList_2511532029(head_2029);
94    }
95 }
```

Gambar 2.2  
Penjelasan program:

Program ini mengimplementasikan tiga jenis operasi penyisipan pada Double Linked List:

- **insertBegin:** Membuat node baru, mengarahkan next-nya ke head lama, kemudian jika head lama tidak null, prev head lama diarahkan ke node baru. Fungsi mengembalikan node baru sebagai head baru. Kompleksitas  $O(1)$ .
- **insertEnd:** Jika list kosong, mengembalikan node baru sebagai head. Jika tidak, melakukan traversal hingga node terakhir, lalu menyambungkan node baru setelah node terakhir dengan mengatur next node terakhir ke node baru dan prev node baru ke node terakhir. Kompleksitas  $O(n)$ .
- **insertAtPosition:** Menyisipkan node pada posisi tertentu. Jika posisi  $\leq 1$ , panggil insertBegin. Jika tidak, traversal hingga node ke-(pos-1). Setelah itu, atur pointer next dan prev node baru serta perbarui pointer node di sekitarnya. Validasi posisi dilakukan untuk mencegah error.

Metode PrintList\_2511532029 mencetak seluruh data dengan format data1 <-> data2 <-> ... menggunakan traversal maju.

Pada main, list awal berisi 2,3,5. Kemudian dilakukan:

1. Insert 1 di depan → list menjadi 1 <-> 2 <-> 3 <-> 5
2. Insert 6 di belakang → 1 <-> 2 <-> 3 <-> 5 <-> 6
3. Insert 4 pada posisi 4 → 1 <-> 2 <-> 3 <-> 4 <-> 5 <-> 6



Output:

```
<terminated> InsertDLL_2511532029 [Java Application] C:\Users\
2029 [Java Application] C:\Users\USER\.p2\pool\plugins\org.ecli
Simpul 6 ditambah diakhir: 1 <-> 2 <-> 3 <-> 5 <-> 6
Tambah node 4 di posisi 4: 1 <-> 2 <-> 3 <-> 4 <-> 5 <-> 6
```

Gambar 2.3

### 3. Program PenelusuranDDL\_2511532029

Program:

```
1 package Pekan6_2511532029;
2
3 public class PenelusuranDDL_2511532029 {
4     static void forwardTraversal(NodeDLL_2511532029 head_2029) {
5         NodeDLL_2511532029 curr_2029 = head_2029;
6         while (curr_2029 != null) {
7             System.out.print(curr_2029.data_2029 + "<->");
8             curr_2029 = curr_2029.next_2029;
9         }
10        System.out.println();
11    }
12
13    static void backwardTraversal_2029(NodeDLL_2511532029 tail_2029) {
14        NodeDLL_2511532029 curr_2029 = tail_2029;
15        while (curr_2029 != null) {
16            System.out.print(curr_2029.data_2029 + "<->");
17            curr_2029 = curr_2029.prev_2029;
18        }
19        System.out.println();
20    }
21 }
22
23 public static void main(String[] args) {
24
25     NodeDLL_2511532029 head_2029 = new NodeDLL_2511532029(1);
26     NodeDLL_2511532029 second_2029 = new NodeDLL_2511532029(2);
27     NodeDLL_2511532029 third_2029 = new NodeDLL_2511532029(3);
28
29     head_2029.next_2029 = second_2029;
30     second_2029.prev_2029 = head_2029;
31     second_2029.next_2029 = third_2029;
32     third_2029.prev_2029 = second_2029;
33
34     System.out.println("Penelusuran maju:");
35     forwardTraversal(head_2029);
36
37     System.out.println("Penelusuran mundur:");
38     backwardTraversal_2029(third_2029);
39
40 }
41
42 }
43
44 }
```

Gambar 2.4

Penjelasan program:

Program ini mendemonstrasikan kemampuan unik Double Linked List yaitu traversal dua arah:

- **forwardTraversal:** Melakukan penelusuran dari node head (awal) hingga node terakhir (tail) dengan mengikuti pointer next\_2029. Setiap node dikunjungi satu per satu dan datanya dicetak. Kompleksitas O(n).
- **backwardTraversal\_2029:** Melakukan penelusuran dari node tail (akhir) hingga node pertama (head) dengan mengikuti pointer prev\_2029. Ini tidak mungkin dilakukan pada Single Linked List tanpa tambahan struktur data. Kompleksitas O(n).

Pada main, dibuat tiga node yang saling terhubung (1 <-> 2 <-> 3). Kemudian dilakukan penelusuran maju dari head (node 1) yang menghasilkan 1<->2<->3<->. Selanjutnya penelusuran mundur dari tail (node 3) yang menghasilkan 3<->2<->1<->.

Output:

```
Console X
<terminated> PenelusuranDLL_2511532029
Penelusuran maju:
1<->2<->3<->
Penelusuran mundur:
3<->
2<->
1<->
```

Gambar 2.5

#### 4. Program HapusDDL\_2511532029

Program:

```
package Pkard_2511532029;

public class ModulDll_2511532029 {
    public static ModulDll_2511532029 delHead_2029(ModulDll_2511532029 head_2029) {
        if (head_2029 == null) {
            return null;
        }
        ModulDll_2511532029 temp_2029 = head_2029;
        head_2029 = head_2029.next_2029;
        if (head_2029 != null) {
            head_2029.prev_2029 = null;
        }
        return head_2029;
    }

    public static ModulDll_2511532029 delLast_2029(ModulDll_2511532029 head_2029) {
        if (head_2029 == null) {
            return null;
        }
        if (head_2029.next_2029 == null) {
            return null;
        }
        ModulDll_2511532029 curr_2029 = head_2029;
        while (curr_2029.next_2029 != null) {
            curr_2029 = curr_2029.next_2029;
        }
        if (curr_2029.prev_2029 != null) {
            curr_2029.prev_2029.next_2029 = null;
        }
        return head_2029;
    }

    public static ModulDll_2511532029 delPos_2029(ModulDll_2511532029 head_2029, int pos_2029) {
        if (head_2029 == null) {
            return head_2029;
        }
        ModulDll_2511532029 curr_2029 = head_2029;
        for (int i = 1; i < pos_2029; i++) {
            curr_2029 = curr_2029.next_2029;
        }
        if (curr_2029 == null) {
            return head_2029;
        }
        if (curr_2029.prev_2029 != null) {
            curr_2029.prev_2029.next_2029 = curr_2029.next_2029;
        }
        if (curr_2029.next_2029 != null) {
            curr_2029.next_2029.prev_2029 = curr_2029.prev_2029;
        }
        return head_2029;
    }

    public static void printList_2029(ModulDll_2511532029 head_2029) {
        ModulDll_2511532029 curr_2029 = head_2029;
        while (curr_2029 != null) {
            System.out.print(curr_2029.data_2029 + "<br>");
            curr_2029 = curr_2029.next_2029;
        }
        System.out.println();
    }

    public static void main(String[] args) {
        ModulDll_2511532029 head_2029 = new ModulDll_2511532029(1);
        head_2029.next_2029 = new ModulDll_2511532029(2);
        head_2029.prev_2029 = null;
        head_2029.next_2029.next_2029 = new ModulDll_2511532029(3);
        head_2029.next_2029.prev_2029 = head_2029;
        head_2029.next_2029.next_2029.next_2029 = new ModulDll_2511532029(4);
        head_2029.next_2029.next_2029.prev_2029 = head_2029;
        head_2029.next_2029.next_2029.next_2029.next_2029 = new ModulDll_2511532029(5);
        head_2029.next_2029.next_2029.next_2029.prev_2029 = head_2029;
        head_2029.next_2029.next_2029.next_2029.next_2029.next_2029 = null;

        System.out.print("List head : ");
        printList_2029(head_2029);

        System.out.print("List head delHead : ");
        head_2029 = delHead_2029(head_2029);
        printList_2029(head_2029);

        System.out.print("List head delLast : ");
        head_2029 = delLast_2029(head_2029);
        printList_2029(head_2029);

        System.out.print("List head delPos : ");
        head_2029 = delPos_2029(head_2029, 2);
        printList_2029(head_2029);
    }
}
```

Gambar 2.6

Penjelasan program:

Program ini mengimplementasikan tiga operasi penghapusan pada Double Linked List:

- **delHead\_2029:** Menghapus node pertama (head). Jika head null, kembalikan null. Jika tidak, pindahkan head ke head.next. Jika head baru tidak null, set prev-nya menjadi null untuk memutuskan hubungan dengan node yang dihapus. Kompleksitas  $O(1)$ .
- **delLast\_2029:** Menghapus node terakhir. Jika head null, kembalikan null. Jika hanya satu node, kembalikan null (list kosong). Traversal hingga node terakhir, lalu set prev.next dari node terakhir menjadi null, sehingga node terakhir terlepas. Kompleksitas  $O(n)$ .

- **delPos\_2029:** Menghapus node pada posisi tertentu. Traversal hingga node ke-pos. Setelah node ditemukan, periksa apakah node memiliki prev (bukan head) dan next (bukan tail), lalu sesuaikan pointer:
  - Jika prev tidak null, set prev.next ke curr.next.
  - Jika next tidak null, set next.prev ke curr.prev.
 Dengan cara ini, node yang dihapus terlepas dari kedua arah.  
 Kompleksitas  $O(n)$ .

Pada main, list awal berisi 1,2,3,4,5. Kemudian dilakukan:

1. delHead\_2029 → menghapus node 1, list menjadi 2,3,4,5.
2. delLast\_2029 → menghapus node terakhir (5), list menjadi 2,3,4.
3. delPos\_2029(head, 2) → menghapus node pada posisi 2 (yaitu node 3), list menjadi 2,4.

Output:

```
<terminated> HapusDLL_2511532029 [Java App
DLL Awal : 1<->2<->3<->4<->5<->
setelah dihapus : 2<->3<->4<->5<->
Setelah node terakhir dihapus2<->3<->4<->
menghapus kode kedua2<->4<->
```

Gambar 2.7

## 2.3 Latihan

### Deskripsi Program:

Program ini mensimulasikan sistem manajemen playlist musik menggunakan struktur data Double Linked List (DLL). Setiap lagu yang ditambahkan akan disimpan dalam playlist dengan kemampuan navigasi dua arah (maju dan mundur). Sistem menyediakan fitur penambahan lagu di akhir playlist, penghapusan lagu pertama, penampilan daftar lagu secara maju dan mundur, serta pencarian lagu berdasarkan judul. Implementasi DLL memungkinkan penelusuran mundur yang tidak dapat dilakukan oleh Single Linked List, sehingga cocok untuk aplikasi pemutar musik dengan fitur "previous" dan "next".

#### 1. Program Lagu\_2511532029

Program:

```
1 package Pekan6_2511532029;
2
3 public class Lagu_2511532029 {
4     // Atribut data lagu
5     String judul_2029;
6     String penyanyi_2029;
7     // Pointer
8     Lagu_2511532029 next_2029;
9     Lagu_2511532029 prev_2029;
10
11     // Constructor
12     public Lagu_2511532029(String judul_2029, String penyanyi_2029) {
13         this.judul_2029 = judul_2029;
14         this.penyanyi_2029 = penyanyi_2029;
15         this.next_2029 = null;
16         this.prev_2029 = null;
17     }
18
19     // Getter & Setter untuk data
20     public String getJudul_2029() { return judul_2029; }
21     public void setJudul_2029(String judul_2029) { this.judul_2029 = judul_2029; }
22
23     public String getPenyanyi_2029() { return penyanyi_2029; }
24     public void setPenyanyi_2029(String penyanyi_2029) { this.penyanyi_2029 = penyanyi_2029; }
25
26     // Getter & Setter untuk pointer
27     public Lagu_2511532029 getNext_2029() { return next_2029; }
28     public void setNext_2029(Lagu_2511532029 next_2029) { this.next_2029 = next_2029; }
29
30     public Lagu_2511532029 getPrev_2029() { return prev_2029; }
31     public void setPrev_2029(Lagu_2511532029 prev_2029) { this.prev_2029 = prev_2029; }
32 }
```

Gambar 2.8

#### a) Deskripsi program Lagu\_2511532029 :

- Kelas ini merepresentasikan node penyimpanan data lagu yang akan disambungkan dalam Double Linked List.
- Berisi atribut judul\_2029 dan penyanyi\_2029 bertipe String, serta atribut public next\_2029 dan prev\_2029 bertipe Lagu\_2511532029 yang berfungsi sebagai referensi ke node berikutnya dan sebelumnya.
- Konstruktor menerima judul dan penyanyi; menginisialisasi kedua pointer ke null.
- Dilengkapi dengan metode getter dan setter untuk mengakses atribut private (enkapsulasi) dan pointer.

## 2. Program RumahSakit\_2511532029

### Program:

```
1  package RumahSakit2511532029;
2  Praktek2029_2511532029/src/Paket2511532029/Lagu_2511532029.java
3
4  public class Musik_2511532029 {
5      // Printer head playlist
6      private Lagu_2511532029 head_2029;
7
8      public Musik_2511532029() {
9          head_2029 = null;
10     }
11
12     // 1. Menambah lagu baru di ANTR playlist
13     public void tambahLagu_2029(String judul_2029, String penyanyi_2029) {
14         Lagu_2511532029 new_node_2029 = new Lagu_2511532029(judul_2029, penyanyi_2029);
15
16         if (head_2029 == null) {
17             head_2029 = new_node_2029;
18             return;
19         }
20
21         // Traversal ke node terakhir
22         Lagu_2511532029 curr_2029 = head_2029;
23         while (curr_2029.next_2029 != null) {
24             curr_2029 = curr_2029.next_2029;
25         }
26
27         // Hubungkan pointer next & prev
28         curr_2029.next_2029 = new_node_2029;
29         new_node_2029.prev_2029 = curr_2029;
30     }
31
32     // 2. Menghapus lagu pertama (head)
33     public void hapusLaguAwal_2029() {
34         if (head_2029 == null) {
35             System.out.println("Playlist kosong, tidak ada lagu yang dihapus.");
36             return;
37         }
38
39         Lagu_2511532029 temp_2029 = head_2029;
40         head_2029 = head_2029.next_2029; // Jarak head ke node berikutnya
41
42         if (head_2029 != null) {
43             head_2029.prev_2029 = null; // Lepaskan koneksi prev ke node yang dihapus
44         }
45         System.out.println("Lagu pertama berhasil dihapus.");
46     }
47
48     // 3. Menampilkan playlist dari awal ke akhir
49     public void tampilMaju_2029() {
50         if (head_2029 == null) {
51             System.out.println("Playlist kosong.");
52             return;
53         }
54
55         Lagu_2511532029 curr_2029 = head_2029;
56         System.out.println("Daftar Playlist (Maju):");
57         while (curr_2029 != null) {
58             System.out.println("- " + curr_2029.judul_2029 + " | " + curr_2029.penyanyi_2029);
59             curr_2029 = curr_2029.next_2029;
60         }
61     }
62
63     // 4. Menampilkan playlist dari akhir ke awal
64     public void tampilMundur_2029() {
65         if (head_2029 == null) {
66             System.out.println("Playlist kosong.");
67             return;
68         }
69
70         // Cari node tail terakhir dahulu
71         Lagu_2511532029 tail_2029 = head_2029;
72         while (tail_2029.next_2029 != null) {
73             tail_2029 = tail_2029.next_2029;
74         }
75
76         // Traversal mundur menggunakan pointer prev_2029
77         Lagu_2511532029 curr_2029 = tail_2029;
78         System.out.println("Daftar Playlist (Mundur):");
79         while (curr_2029 != null) {
80             System.out.println("- " + curr_2029.judul_2029 + " | " + curr_2029.penyanyi_2029);
81             curr_2029 = curr_2029.prev_2029;
82         }
83     }
84
85     // 5. Mencari lagu berdasarkan judul
86     public void cariLagu_2029(String judul_2029) {
87         if (head_2029 == null) {
88             System.out.println("Playlist kosong.");
89             return;
90         }
91
92         Lagu_2511532029 curr_2029 = head_2029;
93         boolean ditemukan_2029 = false;
94
95         while (curr_2029 != null) {
96             if (curr_2029.judul_2029.equalsIgnoreCase(judul_2029)) {
97                 System.out.println("Lagu ditemukan: " + curr_2029.judul_2029 + " oleh " + curr_2029.penyanyi_2029);
98                 ditemukan_2029 = true;
99                 break;
100            }
101            curr_2029 = curr_2029.next_2029;
102        }
103
104        if (!ditemukan_2029) {
105            System.out.println("Lagu dengan judul " + judul_2029 + " tidak ditemukan.");
106        }
107    }
108
109    // Main Menu Interaktif
110    public static void main(String[] args) {
111        Scanner scanner_2029 = new Scanner(System.in);
112        Musik_2511532029 playlist_2029 = new Musik_2511532029();
113        int pilihan_2029 = 0;
114
115        do {
116            System.out.println("===== Playlist Musik NDI: 2511532029 =====");
117            System.out.println("1. Tambah lagu");
118            System.out.println("2. Hapus Lagu Pertama");
119            System.out.println("3. Lihat Playlist (Maju)");
120            System.out.println("4. Lihat Playlist (Mundur)");
121            System.out.println("5. Cari Lagu");
122            System.out.println("6. Keluar");
123            System.out.println("Pilihan: ");
124
125            if (scanner_2029.hasNextInt()) {
126                pilihan_2029 = scanner_2029.nextInt();
127                scanner_2029.nextLine(); // Konsumsi newline
128            } else {
129                System.out.println("Input tidak valid. Masukkan angka.");
130                scanner_2029.nextLine();
131                continue;
132            }
133
134            switch (pilihan_2029) {
135                case 1:
136                    System.out.print("Judul: ");
137                    String judul_input_2029 = scanner_2029.nextLine();
138                    System.out.print("Penyanyi: ");
139                    String penyanyi_input_2029 = scanner_2029.nextLine();
140                    playlist_2029.tambahLagu_2029(judul_input_2029, penyanyi_input_2029);
141                    System.out.println("Lagu berhasil ditambahkan!");
142                    break;
143                case 2:
144                    playlist_2029.hapusLaguAwal_2029();
145                    break;
146                case 3:
147                    playlist_2029.tampilMaju_2029();
148                    break;
149                case 4:
150                    playlist_2029.tampilMundur_2029();
151                    break;
152                case 5:
153                    System.out.print("Masukkan judul lagu yang dicari: ");
154                    String cari_2029 = scanner_2029.nextLine();
155                    playlist_2029.cariLagu_2029(cari_2029);
156                    break;
157                case 6:
158                    System.out.println("Terima kasih telah menggunakan Playlist Musik.");
159                    break;
160                default:
161                    System.out.println("Pilihan tidak tersedia.");
162            }
163        } while (pilihan_2029 != 6);
164        scanner_2029.close();
165    }
166 }
167 }
```

Gambar 2.9

#### a) Deskripsi program RumahSakit\_2511532029

Program ini mengimplementasikan sistem manajemen playlist musik dengan fitur:

1. tambahLagu\_2029: Menambahkan lagu baru di akhir playlist. Memanfaatkan traversal ke node terakhir, kemudian menyambungkan pointer next dan prev. Kompleksitas  $O(n)$ .
2. hapusLaguAwal\_2029: Menghapus lagu pertama (head). Memindahkan head ke node berikutnya, lalu mengatur prev head baru menjadi null. Kompleksitas  $O(1)$ .
3. tampilMaju\_2029: Menampilkan seluruh lagu dari awal ke akhir menggunakan forward traversal. Memanfaatkan pointer next. Kompleksitas  $O(n)$ .
4. tampilMundur\_2029: Menampilkan seluruh lagu dari akhir ke awal menggunakan backward traversal. Pertama, mencari node tail, lalu melakukan traversal mundur menggunakan pointer prev. Fitur ini adalah keunggulan DLL yang tidak dapat dilakukan oleh SSL. Kompleksitas  $O(n)$ .
5. cariLagu\_2029: Mencari lagu berdasarkan judul (case-insensitive). Melakukan forward traversal dan membandingkan judul setiap node. Kompleksitas  $O(n)$ .

#### Output:

```
=== Playlist Musik NIM: 2511532029 ===
1. Tambah Lagu
2. Hapus Lagu Pertama
3. Lihat Playlist (Maju)
4. Lihat Playlist (Mundur)
5. Cari Lagu
6. Keluar
Pilihan: 1
Judul: unand
Penyanyi: unp
Lagu berhasil ditambahkan!

=== Playlist Musik NIM: 2511532029 ===
1. Tambah Lagu
2. Hapus Lagu Pertama
3. Lihat Playlist (Maju)
4. Lihat Playlist (Mundur)
5. Cari Lagu
6. Keluar
Pilihan: 1
Judul: ui
Penyanyi: iu
Lagu berhasil ditambahkan!

=== Playlist Musik NIM: 2511532029 ===
1. Tambah Lagu
2. Hapus Lagu Pertama
3. Lihat Playlist (Maju)
4. Lihat Playlist (Mundur)
5. Cari Lagu
6. Keluar
Pilihan: 3
Daftar Playlist (Maju):
- unand | unp
- ui | iu

=== Playlist Musik NIM: 2511532029 ===
1. Tambah Lagu
2. Hapus Lagu Pertama
3. Lihat Playlist (Maju)
4. Lihat Playlist (Mundur)
5. Cari Lagu
6. Keluar
Pilihan: 4
Daftar Playlist (Mundur):
- ui | iu
- unand | unp
```

Gambar 2.10

```
=== Playlist Musik NIM: 2511532029 ===
1. Tambah Lagu
2. Hapus Lagu Pertama
3. Lihat Playlist (Maju)
4. Lihat Playlist (Mundur)
5. Cari Lagu
6. Keluar
Pilihan: 1
Judul: unand
Penyanyi: unp
Lagu berhasil ditambahkan!

=== Playlist Musik NIM: 2511532029 ===
1. Tambah Lagu
2. Hapus Lagu Pertama
3. Lihat Playlist (Maju)
4. Lihat Playlist (Mundur)
5. Cari Lagu
6. Keluar
Pilihan: 1
Judul: ui
Penyanyi: iu
Lagu berhasil ditambahkan!

=== Playlist Musik NIM: 2511532029 ===
1. Tambah Lagu
2. Hapus Lagu Pertama
3. Lihat Playlist (Maju)
4. Lihat Playlist (Mundur)
5. Cari Lagu
6. Keluar
Pilihan: 2
Lagu pertama berhasil dihapus.

=== Playlist Musik NIM: 2511532029 ===
1. Tambah Lagu
2. Hapus Lagu Pertama
3. Lihat Playlist (Maju)
4. Lihat Playlist (Mundur)
5. Cari Lagu
6. Keluar
Pilihan: 5
Masukkan judul lagu yang dicari: unand
Lagu dengan judul 'unand' tidak ditemukan.

=== Playlist Musik NIM: 2511532029 ===
1. Tambah Lagu
2. Hapus Lagu Pertama
3. Lihat Playlist (Maju)
4. Lihat Playlist (Mundur)
5. Cari Lagu
6. Keluar
Pilihan: 6
Terima kasih telah menggunakan Playlist Musik.
```

Gambar 2.11

## **BAB III**

### **KESIMPULAN**

#### **3.1 Kesimpulan**

Berdasarkan pelaksanaan praktikum implementasi Double Linked List (DLL) dalam Java, dapat disimpulkan bahwa DLL merupakan struktur data dinamis yang unggul dibandingkan Single Linked List karena setiap node memiliki dua pointer (next dan prev), sehingga memungkinkan traversal dua arah (maju dan mundur) tanpa memerlukan struktur tambahan. Seluruh operasi dasar DLL—penyisipan di depan ( $O(1)$ ), belakang ( $O(n)$ ), dan posisi tertentu ( $O(n)$ ); penghapusan di depan ( $O(1)$ ), belakang ( $O(n)$ ), dan posisi tertentu ( $O(n)$ ); serta pencarian ( $O(n)$ )—telah berhasil diimplementasikan. Keberadaan pointer prev menyederhanakan operasi penghapusan node tengah karena akses langsung ke node sebelumnya, berbeda dengan SSL yang memerlukan traversal terpisah.

Penerapan DLL dalam studi kasus Sistem Manajemen Playlist Musik (Musik\_2511532029) membuktikan bahwa konsep linked list dua arah sangat cocok untuk aplikasi yang membutuhkan navigasi maju dan mundur, seperti pemutar musik, browser history, atau fitur undo-redo. Program berhasil mengintegrasikan penambahan lagu di akhir, penghapusan lagu pertama, pencarian lagu, serta fitur unik DLL yaitu penampilan playlist dari akhir ke awal (backward traversal) yang tidak mungkin dilakukan oleh SSL. Penanganan kondisi batas (list kosong, satu elemen, posisi tidak valid) telah diterapkan untuk mencegah NullPointerException. Secara keseluruhan, praktikum ini memperkuat pemahaman tentang manipulasi dua pointer sekaligus dan menjadi fondasi bagi struktur data lanjutan seperti circular doubly linked list serta deque.

#### **3.2 Saran**

1. Penggunaan visualisasi proses penyambungan dan pemutusan dua pointer (next dan prev) melalui animasi atau debug mode akan sangat membantu mahasiswa memahami alur kerja Double Linked List secara lebih intuitif, terutama pada operasi penyisipan dan penghapusan di posisi tengah yang melibatkan perubahan empat referensi pointer sekaligus.
2. Pembagian materi pra-praktikum melalui iLearn tetap perlu dilanjutkan agar mahasiswa memiliki fondasi konseptual yang matang tentang perbedaan SSL dan DLL sebelum menulis kode di laboratorium.

## DAFTAR PUSTAKA

[1] Oracle. "Double Linked List (Java Platform SE 17)". The Java™ Tutorials. 2024. Available:  
<https://docs.oracle.com/javase/tutorial/collections/implementations/>